Despite a diversity in methods, the retrieving components of models predominantly rely on standard approaches, i.e., chunking corpora and encoding with BERT-based retrievers. Although this approach is widely adopted, Nair et al. (2023) highlights a potential shortcoming: contiguous segmentation might not capture the complete semantic depth of the text. Reading extracted snippets from technical or scientific documents may lack important context making them difficult to read or even misleading. (Cohan & Goharian, 2017; Newman et al., 2023; Zhang et al., 2023).

Recursive summarization as Context Summarization techniques provide a condensed view of documents, enabling more focused engagement with the content (Angelidis & Lapata, 2018). The summarization/snippet model by Gao et al. (2023) uses summarizations and snippets of passages, which improves correctness on most datasets but can sometimes be a lossy means of compression. The recursive-abstractive summarization model by Wu et al. (2021) employs task decomposition to summarize smaller text chunks, which are later integrated to form summaries of larger sections. While this method is effective for capturing broader themes, it can miss granular details. LlamaIndex (Liu, 2022) mitigates this issue by similarly summarizing adjacent text chunks but also retaining intermediate nodes thus storing varying levels of detail, keeping granular details. However, both methods, due to their reliance on adjacency for grouping or summarizing adjacent nodes, may still overlook distant interdependencies within the text, which we can find and group with RAPTOR.

3 METHODS

Overview of RAPTOR Building on the idea that long texts often present subtopics and hierarchical structures (Cao & Wang, 2022; Dong et al., 2023b), RAPTOR addresses the issue of semantic depth and connection in reading by building a recursive tree structure that balances broader thematic comprehension with granular details and which allows nodes to be grouped based on semantic similarity not just order in the text.

Construction of the RAPTOR tree begins with segmenting the retrieval corpus into short, contiguous texts of length 100, similar to traditional retrieval augmentation techniques. If a sentence exceeds the 100-token limit, we move the entire sentence to the next chunk, rather than cutting it mid-sentence. This preserves the contextual and semantic coherence of the text within each chunk. These texts are then embedded using SBERT, a BERT-based encoder (`multi-qa-mpnet-base-cos-v1`) (Reimers & Gurevych, 2019). The chunks and their corresponding SBERT embeddings form the leaf nodes of our tree structure.

To group similar text chunks, we employ a clustering algorithm. Once clustered, a Language Model is used to summarize the grouped texts. These summarized texts are then re-embedded, and the cycle of embedding, clustering, and summarization continues until further clustering becomes infeasible, resulting in a structured, multi-layered tree representation of the original documents. An important aspect of RAPTOR is its computational efficiency. The system scales linearly in terms of both build time and token expenditure, making it suitable for processing large and complex corpora. For a comprehensive discussion on RAPTOR's scalability, please refer to the Appendix A.

For querying within this tree, we introduce two distinct strategies: tree traversal and collapsed tree. The tree traversal method traverses the tree layer-by-layer, pruning and selecting the most relevant nodes at each level. The collapsed tree method evaluates nodes collectively across all layers to find the most relevant ones.

Clustering Algorithm Clustering plays a key role in building the RAPTOR tree, organizing text segments into cohesive groups. This step groups related content together, which helps the subsequent retrieval process.

One of the unique aspects of our clustering approach is the use of soft clustering, where nodes can belong to multiple clusters without requiring a fixed number of clusters. This flexibility is essential because individual text segments often contain information relevant to various topics, thereby warranting their inclusion in multiple summaries.

Our clustering algorithm is based on Gaussian Mixture Models (GMMs), an approach that offers both flexibility and a probabilistic framework. GMMs assume that data points are generated from a mixture of several Gaussian distributions.

Given a set of N text segments, each represented as a d-dimensional dense vector embedding, the likelihood of a text vector, $\mathbf{x}$, given its membership in the kth Gaussian distribution, is denoted by $P(\mathbf{x}|k) = \mathcal{N}(\mathbf{x}; \mu_k, \mathbf{\Sigma}_k)$. The overall probability distribution is a weighted combination $P(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x}; \mu_k, \mathbf{\Sigma}_k)$, where π_k signifies the mixture weight for the kth Gaussian distribution.

The high dimensionality of vector embeddings presents a challenge for traditional GMMs, as distance metrics may behave poorly when used to measure similarity in high-dimensional spaces (Aggarwal et al., 2001). To mitigate this, we employ Uniform Manifold Approximation and Projection (UMAP), a manifold learning technique for dimensionality reduction (McInnes et al., 2018). The number of nearest neighbors parameter, $n_neighbors$, in UMAP determines the balance between the preservation of local and global structures. Our algorithm varies $n_neighbors$ to create a hierarchical clustering structure: it first identifies global clusters and then performs local clustering within these global clusters. This two-step clustering process captures a broad spectrum of relationships among the text data, from broad themes to specific details.

Should a local cluster's combined context ever exceed the summarization model's token threshold, our algorithm recursively applies clustering within the cluster, ensuring that the context remains within the token threshold.

To determine the optimal number of clusters, we employ the Bayesian Information Criterion (BIC) for model selection. BIC not only penalizes model complexity but also rewards goodness of fit (Schwarz, 1978). The BIC for a given GMM is $BIC = \ln(N)k - 2\ln(\hat{L})$, where N is the number of text segments (or data points), k is the number of model parameters, and $\hat{L}$ is the maximized value of the likelihood function of the model. In the context of GMM, the number of parameters k is a function of the dimensionality of the input vectors and the number of clusters.

With the optimal number of clusters determined by BIC, the Expectation-Maximization algorithm is then used to estimate the GMM parameters, namely the means, covariances, and mixture weights.

While the Gaussian assumption in GMMs may not perfectly align with the nature of text data, which often exhibits a sparse and skewed distribution, our empirical observations suggest that it offers an effective model for our purpose. We run an ablation comparing GMM Clustering with summarizing contiguous chunks and provide details in Appendix B.

Model-Based Summarization After clustering the nodes using Gaussian Mixture Models, the nodes in each cluster are sent to a language model for summarization. This step allows the model to transform large chunks of text into concise, coherent summaries of the selected nodes. For our experiments, we use `gpt-3.5-turbo` to generate the summaries. The summarization step condenses the potentially large volume of retrieved information into a manageable size. We provide statistics on the compression due to the summarization in Appendix C and the prompt used for summarization in Appendix D.

While the summarization model generally produces reliable summaries, a focused annotation study revealed that about 4% of the summaries contained minor hallucinations. These did not propagate to parent nodes and had no discernible impact on question-answering tasks. For an in-depth analysis of hallucinations, refer to the appendix E.

Querying In this section, we elaborate on the two querying mechanisms employed by RAPTOR: tree traversal and collapsed tree. These methods offer unique ways of traversing the multi-layered RAPTOR tree to retrieve relevant information, each with its own advantages and trade-offs. We provide the pseudocode of both methods in Appendix F. Note that we embed all nodes using SBERT.

The **tree traversal** method first selects the top-k most relevant root nodes based on their cosine similarity to the query embedding. The children of these selected nodes are considered at the next layer and the top-k nodes are selected from this pool again based on their cosine similarity to the query vector. This process is repeated until we reach the leaf nodes. Finally, the text from all selected nodes is concatenated to form the retrieved context. The algorithm's steps are outlined below:

1. Start at the root layer of the RAPTOR tree. Compute the cosine similarity between the query embedding and the embeddings of all nodes present at this initial layer.
2. Choose the top-k nodes based on the highest cosine similarity scores, forming the set S_1.